ECGR 4146/5146 — Introduction to VHDL
Spring 2026

Final Project Report

UART Subsystem with Classification Engine

Group Members:
Alberto Hernandez | Kenneth Le | Erik Contreras Valdivia | Sebastian Navarro

April 30, 2026

Summary

This project implements a complete UART (Universal Asynchronous Receiver/Transmitter) subsystem in VHDL, targeting a 100 MHz clock with 9600 baud serial communication. The system receives serial data, buffers it through a FIFO, writes it into a 32-bit wide block RAM via an FSM controller, runs a character classification engine over the stored data, and transmits the classification results back out over the TX line.

The design is divided into eight modules: a baud rate generator, a UART receiver, a UART transmitter, two FIFO buffers (one on the RX path, one on the TX path), a RAM module, an FSM controller, and a classification engine — all integrated in a top-level module. Every module was developed individually with its own testbench and verified through behavioral simulation in Vivado 2024.2 before being connected at the top level.

The classification engine categorizes each received byte into one of four classes: uppercase letter (0x01), lowercase letter (0x02), digit (0x03), or other (0x00). Results are streamed back to the transmitter and sent serially over the TX line. A full integration testbench sends 128 bytes through the RX line and verifies all 128 classification results captured from the TX output.

Module 1: UART Subsystem

The UART subsystem handles all serial communication. It consists of three components: the baud rate generator, the receiver, and the transmitter. All three share the same 100 MHz system clock and communicate using a 16× oversampling scheme at 9600 baud.

1.1 Baud Rate Generator (baud_gen.vhd)

The baud rate generator produces a single-cycle tick pulse every M clock cycles, where M is set by a generic parameter. For 9600 baud with a 100 MHz clock and 16× oversampling, M = 651 (100,000,000 / (9600 × 16) ≈ 651). The module uses a simple counter that wraps at M−1 and asserts the tick output for exactly one clock cycle when it rolls over. Both the receiver and transmitter connect to this tick signal.

Port	Direction	Width	Description
clk	in	1-bit	100 MHz system clock
reset	in	1-bit	Active-HIGH synchronous reset
tick	out	1-bit	Baud tick — HIGH for 1 cycle every 651 cycles (16× 9600 baud)

1.2 UART Receiver (uart_rx.vhd)

The receiver uses a 4-state FSM (idle, start, data, stop) to deserialize incoming serial frames. It waits for the RX line to fall low (start bit), then aligns itself to the middle of the start bit by waiting 7 ticks. From there it samples each of the 8 data bits at tick 15 (the midpoint of each bit period), shifting them into an 8-bit register LSB-first. After all 8 data bits are received, it waits through the stop bit period and asserts rx_done for one clock cycle to signal that a valid byte is available on dout.

Port	Direction	Width	Description
clk	in	1-bit	100 MHz system clock
reset	in	1-bit	Active-HIGH synchronous reset
rx	in	1-bit	Serial input line (idle HIGH)
s_tick	in	1-bit	Baud tick from baud_gen
rx_done	out	1-bit	Pulses HIGH for 1 cycle when byte is received
dout	out	8-bit	Received byte — valid when rx_done is HIGH

1.3 UART Transmitter (uart_tx.vhd)

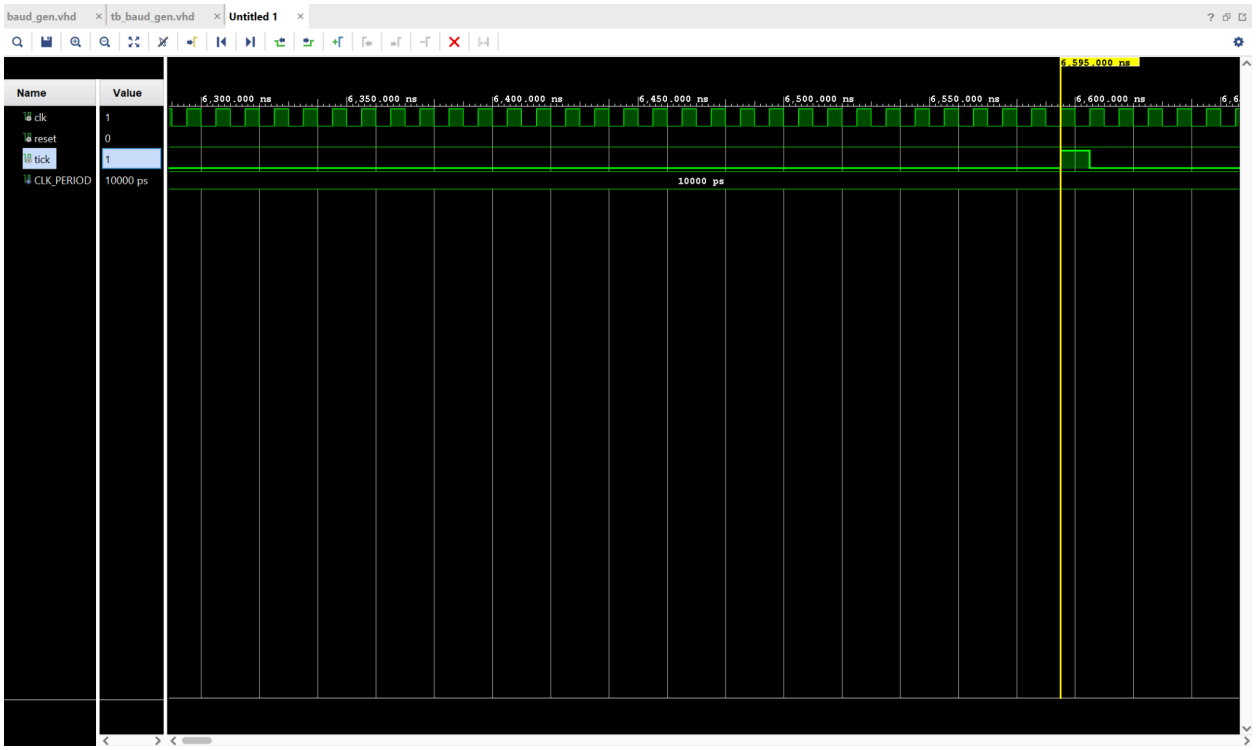
The transmitter mirrors the receiver using the same 4-state FSM structure (idle, start, data, stop). When tx_start pulses HIGH, it loads the byte from din, transmits a start bit (LOW), then shifts out all 8 data bits LSB-first at the baud rate, followed by a stop bit (HIGH). It asserts tx_done for one cycle after the stop bit completes. The TX line idles HIGH when the transmitter is not active.

Port	Direction	Width	Description
clk	in	1-bit	100 MHz system clock
reset	in	1-bit	Active-HIGH synchronous reset
tx_start	in	1-bit	Pulse HIGH to begin transmission
s_tick	in	1-bit	Baud tick from baud_gen
din	in	8-bit	Byte to transmit
tx_done	out	1-bit	Pulses HIGH for 1 cycle when byte is fully sent
tx	out	1-bit	Serial output line (idle HIGH)

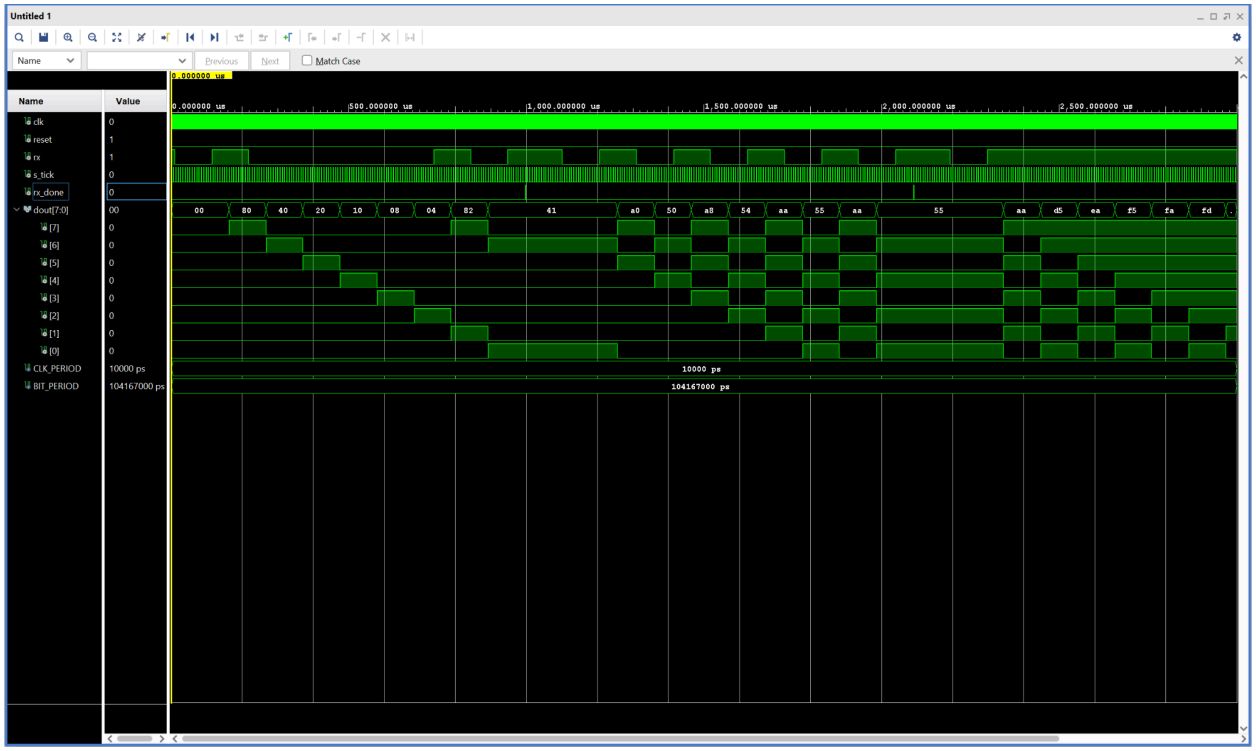
1.4 Testbenches

Each UART component has its own dedicated testbench. The baud generator testbench (tb_baud_gen.vhd) runs the counter for 200 µs and verifies tick spacing is approximately 6.51 µs. The receiver testbench (tb_uart_rx.vhd) drives the RX line with four known byte patterns (0x41, 0x55, 0xFF, 0x00) using a procedure that generates correct UART framing at 9600 baud, then checks dout after each frame. The transmitter testbench (tb_uart_tx.vhd) applies tx_start with two test bytes (0x55 and 0xA3) and verifies the serial output. A separate loopback testbench (tb_loopback.vhd) connects the receiver directly to the transmitter through a register and verifies three bytes are correctly echoed.

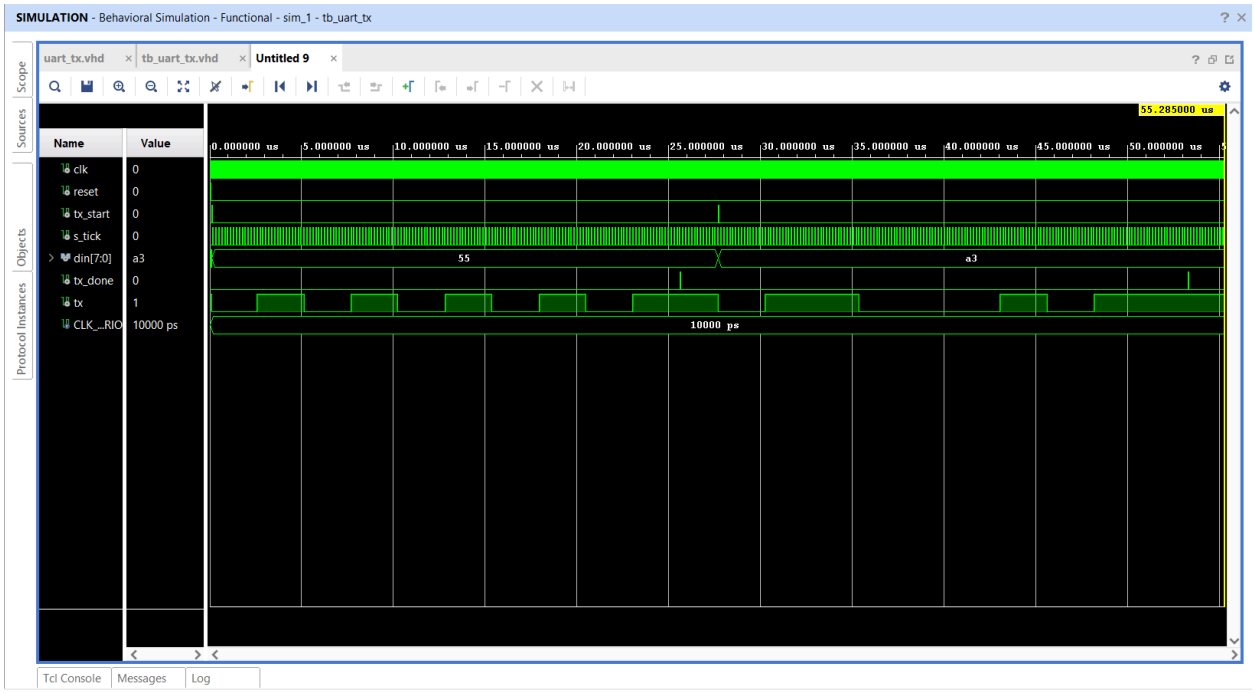
1.5 Waveforms



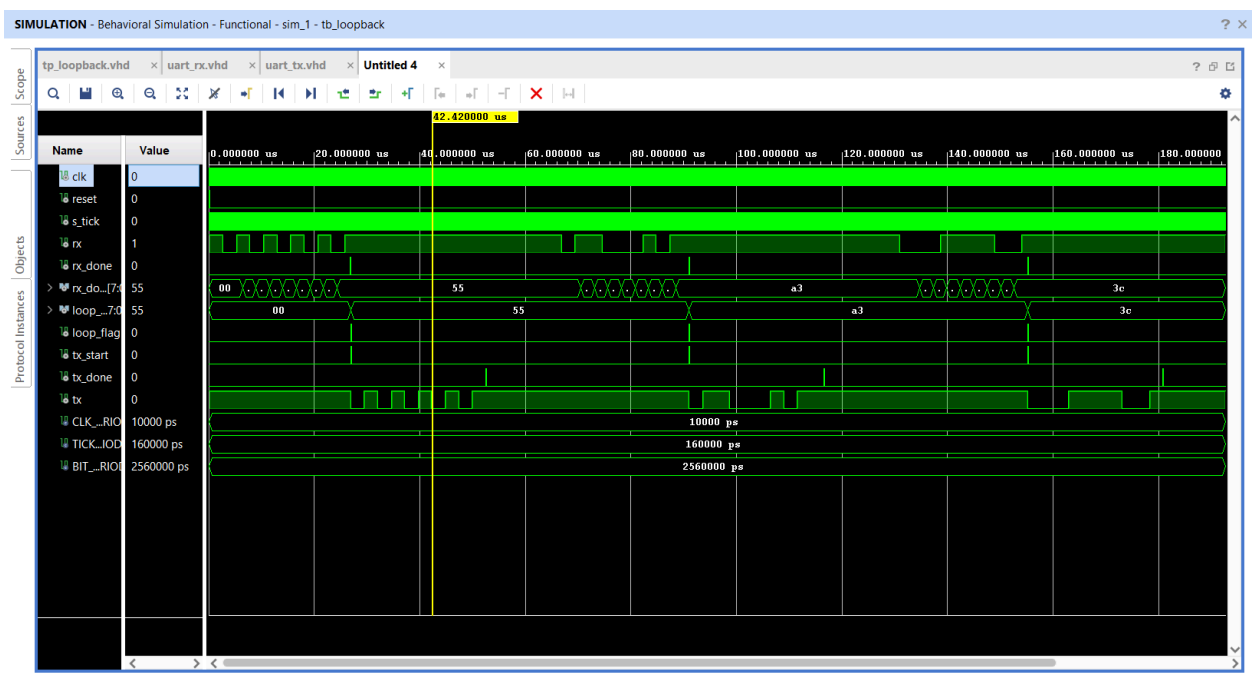
baud_gen_tick



uart_rx_receive



uart_tx_transmit



loopback_rx_tx

Module 2: FIFO-Based Interface Design

Two FIFO buffer instances are used in the design, one on the RX path (rx_fifo) and one on the TX path (tx_fifo). Both are instances of the same parameterized FIFO entity (fifo.vhd). The RX FIFO decouples the receiver from the FSM controller, allowing bytes to arrive at any time without the FSM needing to be ready immediately. The TX FIFO buffers classification results from the engine before the transmitter drains them one at a time.

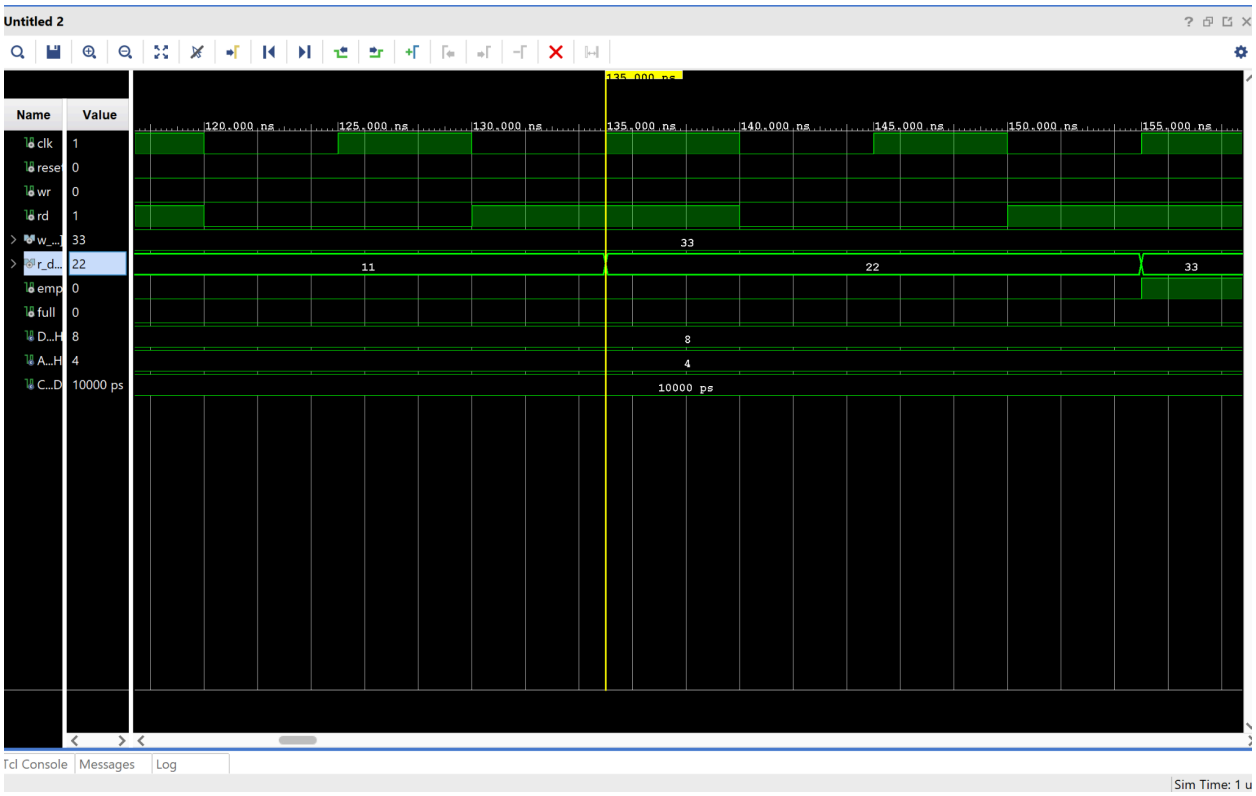
The FIFO uses a circular buffer with separate read and write pointers. The empty and full flags are derived by comparing these pointers. The implementation supports simultaneous read and write in the same clock cycle, if a write comes in while the FIFO is full and a read is also happening, the write is permitted because the read frees a slot. The RX FIFO uses a 4-bit address width (16 entries), while the TX FIFO uses an 8-bit address width (255 usable slots) to hold all 128 classification results without stalling.

Port	Direction	Width	Description
clk	in	1-bit	100 MHz system clock
reset	in	1-bit	Active-HIGH synchronous reset — clears FIFO
wr	in	1-bit	Write enable
rd	in	1-bit	Read enable
w_data	in	8-bit	Data to write
r_data	out	8-bit	Data read from FIFO head
empty	out	1-bit	HIGH when FIFO has no data
full	out	1-bit	HIGH when FIFO is full

2.1 Testbench

The FIFO testbench (tb_fifo.vhd) covers eight test scenarios: reset behavior, writing three bytes and checking empty deasserts, reading them back in order with value assertions, attempting a read on an empty FIFO, filling to capacity (15 entries with a 4-bit pointer), attempting a write on a full FIFO, reading one entry to un-full the FIFO, and simultaneous read/write. All assertions use severity error so failures are immediately visible in the simulation log.

2.2 Waveforms



fifo_readwrite

Module 3: RAM Module

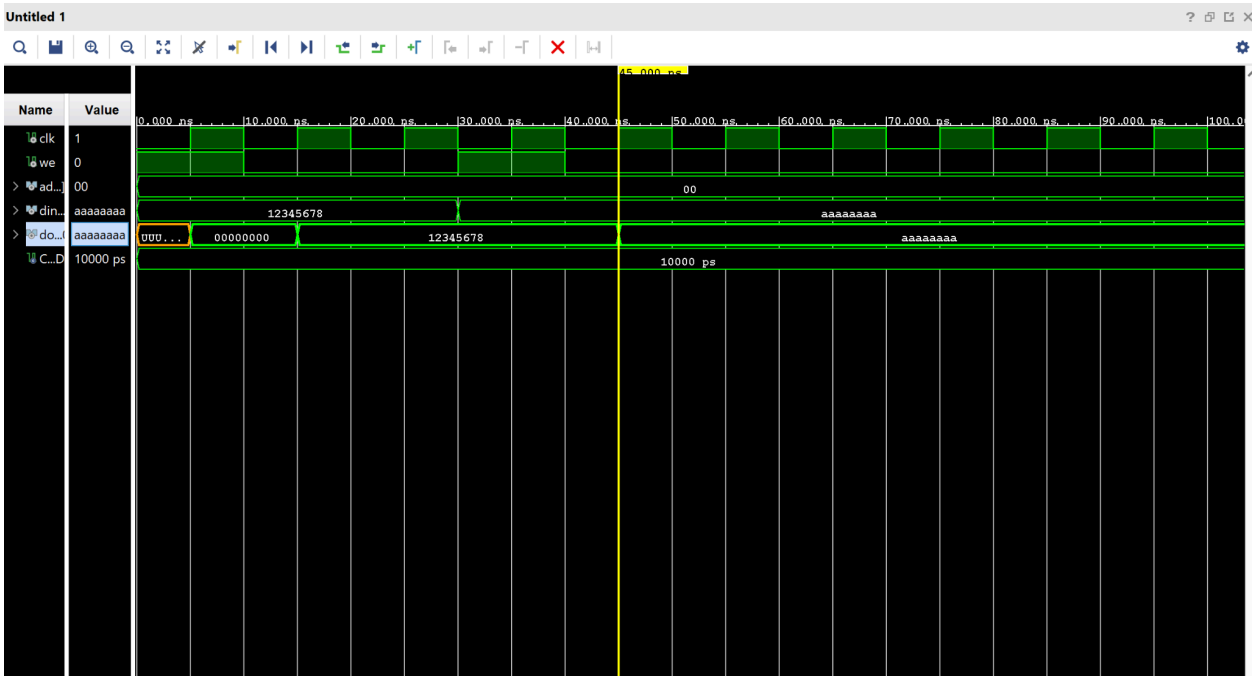
The RAM module (ram_module.vhd) is a 32-bit wide synchronous block RAM with 32 rows (5-bit address space). Both reads and writes are registered on the rising clock edge, giving reads a one-cycle latency. The write enable (we) gates the write operation, and the read always outputs the value at the current address registered on the previous cycle. The RAM is implemented using a VHDL array type to infer a block RAM resource in synthesis.

Port	Direction	Width	Description
clk	in	1-bit	100 MHz system clock
we	in	1-bit	Write enable — active HIGH
addr	in	5-bit	Row address (0–31)
din	in	32-bit	Data to write
dout	out	32-bit	Registered read output (1-cycle latency)

3.1 Testbench

The RAM testbench (tb_ram.vhd) writes distinct 32-bit patterns to five addresses, reads them back after one cycle of latency, asserts the correct values, then overwrites address 0 and verifies the update. It also confirms all five original values before the overwrite test to catch any corruption.

3.2 Waveforms



ram_readwrite

Module 4: Classification Engine

The classification engine (classification_engine.vhd) reads all 32 rows from the RAM, classifies each of the four bytes in every 32-bit word, and streams the results out one byte at a time via result and result_valid. Classification is done with a pure combinational function that maps byte ranges to codes: uppercase letters A–Z map to 0x01, lowercase a–z to 0x02, digits 0–9 to 0x03, and everything else to 0x00.

The engine uses a 6-state FSM: IDLE waits for the start pulse, READ_ADDR presents the current RAM address, WAIT_RAM accounts for the one-cycle read latency, CLASSIFY unpacks and classifies all four bytes in the current word, OUTPUT streams them out one at a time, and DONE_ST pulses the done signal. After done asserts, the engine returns to IDLE and can be re-triggered.

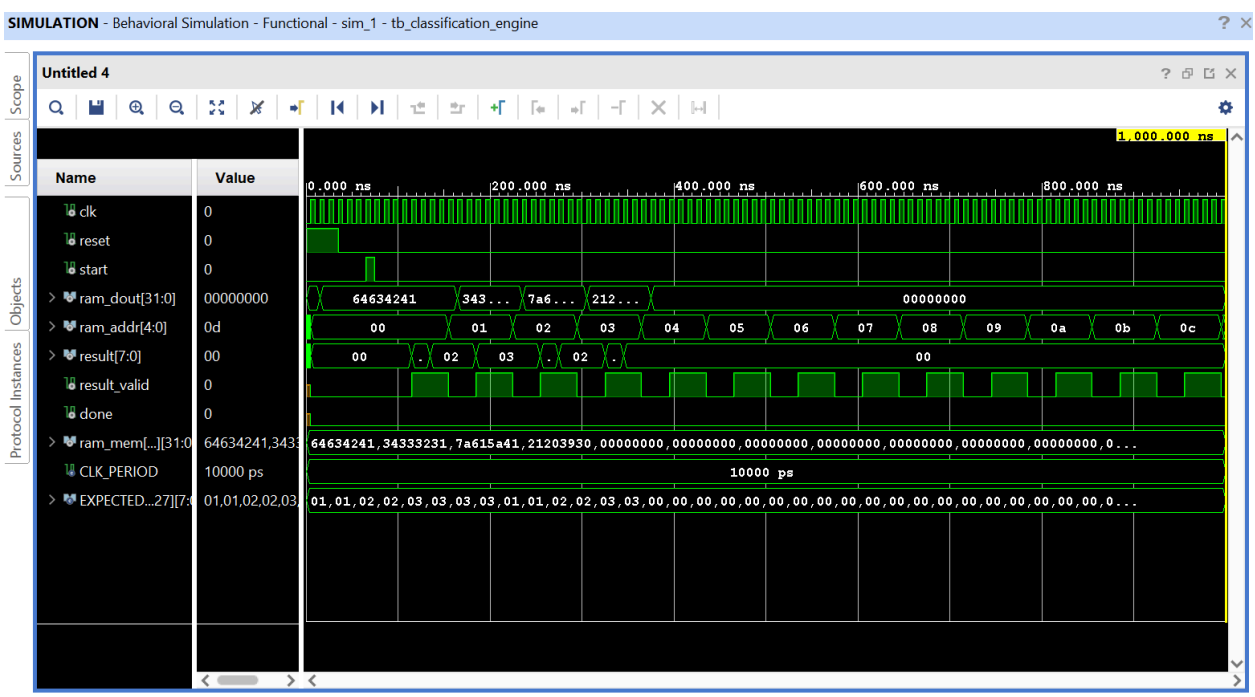
Port	Direction	Width	Description
clk	in	1-bit	100 MHz system clock
reset	in	1-bit	Active-HIGH synchronous reset

start	in	1-bit	Pulse HIGH to begin classification (from fsm_done)
ram_dout	in	32-bit	Data from RAM module
ram_addr	out	5-bit	Read address driven to RAM
result	out	8-bit	One classification result byte
result_valid	out	1-bit	Pulses HIGH for 1 cycle per result byte
done	out	1-bit	Pulses HIGH when all 128 results produced

4.1 Testbench

The classification engine testbench (tb_classification_engine.vhd) preloads a 32-entry RAM model with four test words covering all classification cases: mixed-case ASCII letters (A, B, c, d), digits (1, 2, 3, 4), boundary values at the edges of each range (A, Z, a, z, 0, 9), and non-alphanumeric characters (space, exclamation mark). It collects all 128 result bytes and compares each one against a precomputed expected array, reporting pass/fail per result with the actual and expected hex values. It also tests synchronous reset mid-operation to verify no spurious outputs follow.

4.2 Waveforms



classification_results

Module 5: Integrated UART System

The top-level module (uart_top.vhd) wires all eight components together into the complete data path. A phase signal controls which module drives the RAM address: during the receive phase (phase = 0), the FSM owns the RAM for writing; after fsm_done asserts, the phase flips to 1 and the classification engine takes over for reading. The phase returns to 0 after ce_done.

A three-state TX controller FSM (TX_IDLE, TX_LOAD, TX_WAIT) drains the TX FIFO one byte at a time. When the FIFO is not empty, it pops the head byte and asserts tx_start to the transmitter, then waits for tx_done before going back for the next byte.

5.1 FSM Controller (uart_fsm.vhd)

The FSM controller reads bytes from the RX FIFO and packs them into 32-bit words before writing to RAM. It uses a 6-state FSM: IDLE waits for data in the FIFO, READ_BYTE latches the current FIFO head and pulses fifo_rd to advance the pointer, PACK places the byte into its lane in the 32-bit accumulator, WRITE_RAM asserts ram_we for one cycle, DONE_PULSE asserts fsm_done for one cycle after row 31 is written, and DONE is a terminal state. Bytes are packed little-endian: the first byte received goes into bits [7:0], the second into [15:8], and so on.

State	Action
IDLE	Wait for fifo_empty = 0
READ_BYTE	Latch fifo_dout into byte_buf, pulse fifo_rd
PACK	Place byte_buf into current lane of word_reg; go to WRITE_RAM if lane 3 filled, else back to IDLE
WRITE_RAM	Assert ram_we for one cycle; increment RAM address; go to DONE_PULSE if addr = 31
DONE_PULSE	Assert fsm_done for one cycle
DONE	Terminal — stay idle

5.2 System Integration Diagram

The data flow through the system is as follows:

RX → UART Receiver → rx_fifo → uart_fsm → RAM → classification_engine → tx_fifo → UART Transmitter → TX

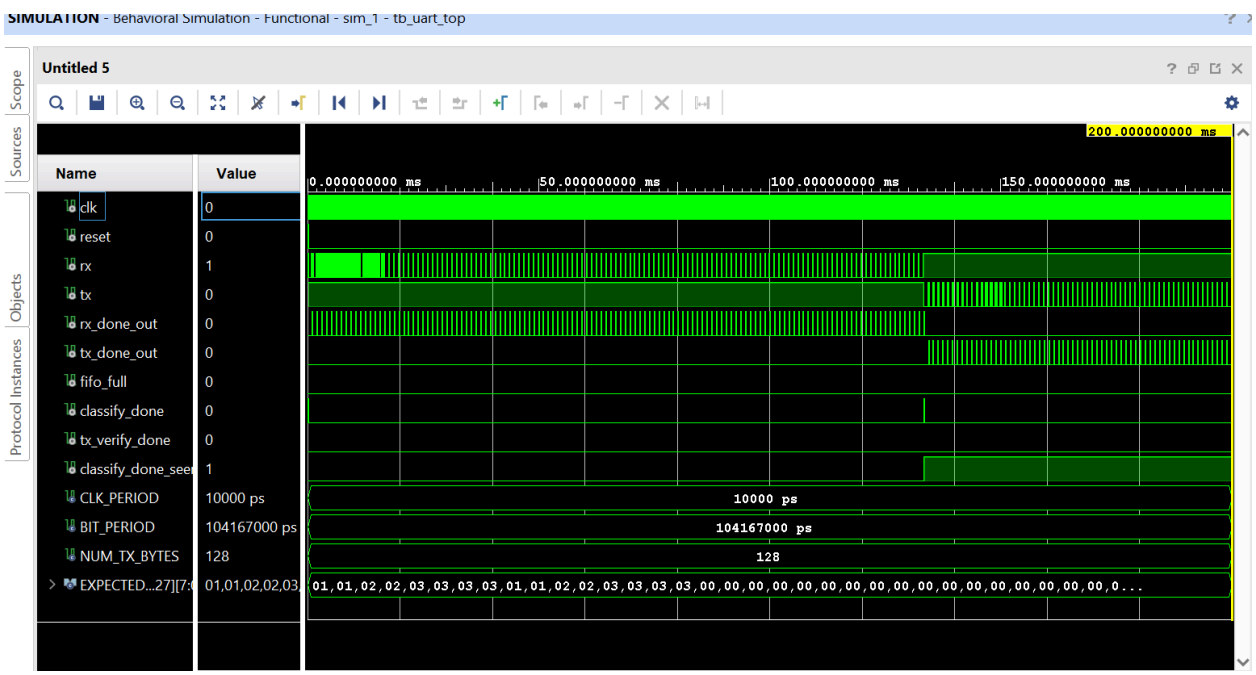
5.3 Testbench

The integration testbench (tb_uart_top.vhd) sends 128 bytes through the RX line: 16 typed characters (ABcd1234EFgh5678) followed by 112 null bytes to fill all 32 RAM rows. It then monitors the TX line and samples all 128 transmitted bytes, comparing each one against the expected classification code. A

separate TX verification process runs concurrently, aligning itself to the center of each bit cell to sample reliably. The testbench reports a full pass/fail summary at the end with byte-level granularity.

Input Byte	Character	Expected Output
0x41	A	0x01 (uppercase)
0x42	B	0x01 (uppercase)
0x63	c	0x02 (lowercase)
0x64	d	0x02 (lowercase)
0x31–0x34	1–4	0x03 (digit)
0x45, 0x46	E, F	0x01 (uppercase)
0x67, 0x68	g, h	0x02 (lowercase)
0x35–0x38	5–8	0x03 (digit)
0x00	null	0x00 (other)

5.4 Waveforms



uart_top_full

Challenges and Solutions

FIFO Pointer Collision

The most subtle issue in the FIFO design was determining when the buffer is full versus empty. Both conditions result in the read and write pointers being equal, so a simple comparison is not enough to distinguish them. The solution was to make the FIFO hold one fewer entry than the address space allows. The full condition is detected when incrementing the write pointer would make it equal to the read pointer, rather than when they are currently equal. This ensures empty and full are never ambiguous.

FIFO Read Latency in the FSM

The FSM controller operates against a FWFT (First Word Fall Through) FIFO, meaning the current head byte is always visible on r_data without needing a read strobe first. However, pulsing fifo_rd advances the pointer on the next clock edge, which means the byte at r_data changes one cycle after the strobe. To avoid reading a stale or already-advanced value, the FSM latches fifo_dout into an internal byte_buf register in the READ_BYTE state, then processes that latched value in PACK. This guarantees the byte used for packing is the one that was valid when the read strobe fired.

RAM Address Muxing Between FSM and Classifier

Both the FSM (writing) and the classification engine (reading) need access to the RAM address port, but they operate in different phases. The solution was a registered phase signal that switches after fsm_done asserts and returns after ce_done asserts. The top-level uses this signal to mux the RAM address and write-enable between the two modules. This keeps both modules simple and self-contained, neither needs to know about the other.

TX FIFO Depth

The classification engine produces 128 result bytes (32 rows × 4 bytes each) faster than the transmitter can send them at 9600 baud. With a shallow FIFO the engine would stall waiting for the transmitter, or results would be dropped. The TX FIFO was sized with an 8-bit address width (255 usable entries) to comfortably buffer all 128 results in one shot, letting the classifier run at full speed and the transmitter drain at its own pace.

AI Tool Usage

AI assistance was used in three areas during this project. First, it helped identify a subtle timing issue in the FSM, specifically that fifo_dout would already reflect the next byte by the time the PACK state executed, which led to the byte_buf latching fix in READ_BYTE. Second, it was used to draft and maintain INTERFACE_AGREEMENT.md, which served as the team's shared reference for port names, signal widths, and active levels across all modules. Third, it was used to review VHDL modules and testbenches for correctness and consistency.

Conclusion

The UART subsystem with classification engine was successfully designed, implemented, and verified in VHDL using Vivado 2024.2. All seven core modules compile, pass their individual testbenches, and function correctly when integrated at the top level. The system receives serial data at 9600 baud, stores it to RAM, classifies each byte, and transmits the classification results back over the TX line.

The project covered a range of digital design concepts including FSM design, synchronous FIFO implementation, synchronous block RAM inference, multi-module integration, and serial communication protocols. The biggest design lessons came from the timing interactions between modules, particularly the FIFO read latency issue and the RAM address mux strategy, both of which required careful thought about which module owns which signal and when.

If extended, the system could benefit from a configurable baud rate set at runtime, error detection on the RX path (framing and parity checks), and a more complex classification engine that handles multi-byte patterns. The current design provides a solid foundation for any of those additions.